

than c . We then find the quotient and remainder when dividing b/c . If we say that b/c is q_2 , with remainder d , we can write our result as follows:

$$b = q_2 \times c + d$$

Again, d will be smaller than both b and c , since it's a remainder. If you look at our two equations here, you can start to see a pattern: we're working our way through the alphabet, shifting terms to the left every time. We started with a , b , and c , and then we had b , c , and d . You can see this pattern continue in our next step, in which we divide c/d , and call the quotient q_3 and the remainder e .

$$c = q_3 \times d + e$$

We can continue this process, proceeding as far as we need through the alphabet, until the remainder is equal to zero. Remember that remainders are always smaller than the numbers that were divided to get them, so c is smaller than a and b , d is smaller than b and c , e is smaller than c and d , and so on. This means that at every step, we're working with smaller and smaller integers, so we must eventually get to zero. When we get a zero remainder, we stop the process, and we know that the last nonzero remainder is the greatest common divisor. For example, if we find that e is zero, then d is the greatest common divisor of our original two numbers.

Implementing Euclid's Algorithm in Python

We can implement this algorithm in Python quite easily, as shown in [Listing 2-1](#).

```
def gcd(x,y):
    larger = max(x,y)
    smaller = min(x,y)

    remainder = larger % smaller

    if(remainder == 0):
        return(smaller)
```